

# IoT-Based Smart City Parking Management System

**Prepared For**

Smart-Internz

Internet of Things (IoT) Guided Project

**By**

Sanika Sambhaji Patil

D Y Patil Agriculture and Technical University, Talsande

## **Abstract**

Urbanization has driven a sharp rise in vehicular traffic, leading to severe parking congestion, fuel wastage, and increased carbon emissions in smart cities. This project presents a web-based Smart City Parking Management System that simulates a real-time, automated parking infrastructure in which individual parking slots act as digital IoT nodes. The application monitors space availability, automates vehicle check-in and slot allocation, updates a live sensor-style grid map, and performs automatic micro-billing on checkout. By eliminating manual searching for parking, the system offers a scalable blueprint for efficient urban space utilization.

# 1. Introduction

## 1.1 Project Overview

As cities grow denser, finding an open parking spot has become a daily source of congestion, wasted fuel, and driver frustration. Conventional parking facilities rely on manual attendants or static signage, offering drivers no visibility into real-time space availability and no automated way to track occupancy or bill accurately for usage time.

This project, the Smart City Parking Management System, is a web application built on the Python Flask framework that models a fully automated parking facility. Each of the facility's parking bays is treated as a simulated IoT sensor node, with its own live status, assigned vehicle, and entry timestamp. The system gives operators and drivers a single live dashboard showing exactly which bays are free or occupied, automatically assigns incoming vehicles to the nearest available bay, and calculates a checkout invoice based on actual time parked.

Core functionality includes:

- Live Occupancy Monitoring — a real-time grid map showing every bay as available or occupied.
- Automated Slot Allocation — incoming vehicles are assigned the first free bay automatically.
- Digital Vehicle Check-In — a simple form captures the vehicle registration number at entry.
- Automated Checkout & Billing — elapsed parking duration is calculated and an invoice generated instantly.
- Status Dashboard — a live summary of total available and occupied bays for facility operators.

## 1.2 Purpose / Objectives

The project sets out to demonstrate how lightweight web and simulated-IoT technology can modernize urban parking management:

- Reducing Search Time — eliminating the need for drivers to manually search for an open spot.
- Automating Operations — removing manual logging of entries, exits, and payments.
- Accurate, Transparent Billing — calculating fares automatically from actual time parked, removing disputes over manual estimates.
- Operational Visibility — giving facility managers a live, at-a-glance view of occupancy levels.
- Scalability — providing a blueprint that can be extended from a handful of bays to a full multi-level structure or networked city-wide system.

## 2. Literature Survey

### 2.1 Existing Problem

Most conventional parking facilities still depend on boom-gate ticketing, manual attendants, or simple first-come-first-served signage. These approaches give drivers no advance knowledge of space availability, often resulting in vehicles circling a facility or city block in search of a free bay. Billing is typically calculated manually from a printed ticket, which is slow, error-prone, and offers no real-time visibility to either the driver or the facility operator.

Existing automated solutions that do exist are often costly, requiring physical sensor hardware (ultrasonic or infrared sensors per bay) and dedicated networking infrastructure, which limits adoption to large commercial facilities. There is a clear need for a lightweight, software-driven approach that can demonstrate the same automated experience without the upfront hardware investment, and that can later be connected to real sensors as a facility scales up.

### 2.2 Related Work

Research into smart parking has generally followed two tracks: sensor-based occupancy detection (using ultrasonic, infrared, or magnetic sensors embedded in each bay) and camera/computer-vision-based detection (using image recognition to classify bays as occupied or free from overhead or angled cameras). Sensor-based systems tend to be highly accurate but require per-bay hardware installation and maintenance, while vision-based systems reduce hardware cost but are sensitive to lighting and camera placement.

A growing body of smart-city literature also explores software-simulated IoT models, where backend logic mimics what physical sensors would report, allowing rapid prototyping and demonstration of the user experience and business logic (allocation, billing, alerts) before committing to hardware deployment. This project follows that simulated-IoT approach, using an in-memory data model to stand in for a physical sensor network.

### 2.3 Problem Statement Definition

Conventional parking management is manual, opaque to drivers, and inconsistent in billing accuracy. This project addresses that gap by building a Smart City Parking Management System that automatically tracks bay occupancy, allocates incoming vehicles to the nearest free bay, and generates accurate, time-based invoices at checkout — all through a simple web dashboard that requires no specialized hardware to demonstrate.

## 3. Ideation & Proposed Solution

### 3.1 Empathy Map Canvas

The key stakeholders for a smart parking facility are drivers seeking a spot and facility operators managing day-to-day occupancy and billing.

- Says: Drivers say they want to know before arriving whether a spot is free; operators say manual logs make it hard to track occupancy and revenue accurately.

- **Thinks:** Drivers think about minimizing time spent circling for parking; operators think about reducing disputes over billing and improving turnover of bays.
- **Does:** Drivers search visually for open spots and queue at entry gates; operators manually note entry times and calculate fares on exit.
- **Feels:** Drivers feel frustrated by unpredictable wait times; operators feel pressure to keep accurate records while serving vehicles quickly.

### 3.2 Ideation & Brainstorming

- **Real-Time Occupancy Grid** — a single dashboard view of every bay's status, refreshed on each action.
- **Automated Nearest-Bay Allocation** — a simple, low-complexity algorithm to assign the first available bay to an arriving vehicle.
- **Digital Check-In Form** — a lightweight form capturing only the vehicle registration number, minimizing friction at entry.
- **Time-Based Micro-Billing** — a transparent fare formula combining a fixed base charge with a per-minute rate.
- **Status Feed Banner** — a persistent summary of available versus occupied bays for quick operator reference.
- **Simulated IoT Sensor Layer** — an in-memory data model standing in for physical hardware sensors, enabling fast prototyping and future hardware integration.

The team converged on a Flask-based web application following the Model-View-Controller pattern, using a simulated sensor data layer that can later be swapped for real hardware sensor feeds without changing the allocation or billing logic.

## 4. Requirement Analysis

### 4.1 Functional Requirements

The system must maintain a live record of every parking bay's status (available or occupied), the registration number of any vehicle currently parked, and its entry timestamp. On vehicle check-in, the system must scan for the first available bay and assign the vehicle to it, rejecting the request gracefully if the facility is full.

On checkout, the system must look up the relevant bay, calculate the elapsed parking duration from the recorded entry time, apply a fare formula combining a fixed base rate and a per-minute charge, and present the driver with an itemized invoice before freeing the bay for the next vehicle. The dashboard must display, at all times, the total count of available and occupied bays and a visual grid showing the status of every individual bay.

### 4.2 Non-Functional Requirements

- **Performance:** dashboard and allocation actions should respond within a second under normal load for a small-to-medium facility.
- **Usability:** check-in and checkout forms should require minimal input and be understandable without training.

- **Reliability:** the system should handle edge cases gracefully, such as a full facility or an invalid checkout request, without crashing.
- **Scalability:** the data model should support easily increasing the number of bays or extending to multiple facilities.
- **Maintainability:** the backend (Controller/Model) and frontend (View) should remain cleanly separated so either can be updated independently.
- **Portability:** the application should run on any platform supporting Python and Flask, including Windows, Linux, and containerized environments.
- **Extensibility:** the simulated sensor layer should be replaceable with real hardware or networked IoT sensor feeds without redesigning the allocation or billing logic.

## 5. Project Design

### 5.1 Data Flow & User Stories

Level 0 (Context) DFD: Driver / Operator Actions → Parking Management System → Updated Dashboard & Invoice.

Level 1 DFD components: a Check-In Form (input) captures the vehicle registration number; an Allocation Module scans the bay data for the first available slot; a Status Module aggregates occupancy counts for the dashboard; a Checkout Module computes elapsed time and fare; output is delivered as an updated grid map, a status banner, and (on checkout) a billing message.

Representative user stories:

- As a driver, I want to be automatically assigned a free bay so I don't have to search for one.
- As a facility operator, I want a live dashboard showing how many bays are free so I can manage capacity.
- As a facility operator, I want an automatically calculated invoice at checkout so billing is accurate and dispute-free.

### 5.2 Solution Architecture

The system follows the Model-View-Controller (MVC) pattern within the Flask ecosystem:

- **Model (Data Layer)** — an in-memory data structure simulating sensor hardware, tracking each bay's status, assigned vehicle, and entry time.
- **View (Frontend Layer)** — an HTML5 and CSS3 dashboard template, dynamically rendered to reflect live occupancy data.
- **Controller (Backend Layer)** — the Flask application logic that defines the check-in and checkout endpoints, performs slot-assignment, and computes checkout invoicing.

## 6. Project Planning & Scheduling

### 6.1 Technical Architecture

The system combines a lightweight backend with a responsive dashboard frontend across the following components:

- Data Layer — an in-memory dictionary structure simulating per-bay sensor status, vehicle registration, and entry time.
- Backend Layer — a Flask application defining routes for the dashboard, vehicle check-in, and checkout/billing.
- Frontend Layer — an HTML/CSS dashboard rendered via the Jinja2 templating engine, showing a live status banner and grid map.
- Billing Logic — a fare calculation module applying a fixed base charge plus a per-minute rate based on elapsed parking duration.

### 6.2 Sprint Planning & Estimation

| Sprint   | Duration | Start      | End        | Goals / Deliverables                                | Estimate (PD) |
|----------|----------|------------|------------|-----------------------------------------------------|---------------|
| Sprint 1 | 1 wk     | 2025-03-16 | 2025-03-22 | Requirement analysis, environment setup, MVC design | 6             |
| Sprint 2 | 1 wk     | 2025-03-23 | 2025-03-29 | Build in-memory data model & sensor simulation      | 8             |
| Sprint 3 | 2 wks    | 2025-03-30 | 2025-04-12 | Develop Flask routes for check-in / checkout logic  | 12            |
| Sprint 4 | 1 wk     | 2025-04-13 | 2025-04-19 | Build dashboard UI, grid map & status feed          | 8             |
| Sprint 5 | 1 wk     | 2025-04-20 | 2025-04-26 | Implement billing/invoicing logic                   | 6             |
| Sprint 6 | 1 wk     | 2025-04-27 | 2025-05-03 | Testing, bug fixing & documentation                 | 6             |

### 6.3 Sprint Delivery Schedule

| Sprint   | Start      | End        | Milestone                                   | Deliverables                          |
|----------|------------|------------|---------------------------------------------|---------------------------------------|
| Sprint 1 | 2025-03-16 | 2025-03-22 | Project initiation & architecture finalized | Requirements doc, MVC design          |
| Sprint 2 | 2025-03-23 | 2025-03-29 | Core data model ready                       | Simulated sensor data structure       |
| Sprint 3 | 2025-03-30 | 2025-04-12 | Backend routing complete                    | /park and /checkout endpoints working |
| Sprint 4 | 2025-04-13 | 2025-04-19 | Frontend dashboard complete                 | Live grid map, status                 |

|          |            |            |                            |                                        |
|----------|------------|------------|----------------------------|----------------------------------------|
|          |            |            |                            | feed UI                                |
| Sprint 5 | 2025-04-20 | 2025-04-26 | Billing module complete    | Automated invoice generation           |
| Sprint 6 | 2025-04-27 | 2025-05-03 | Final testing & deployment | Tested application, user documentation |

## 7. Key Functional Modules

### 7.1 Real-Time Initialization & Baseline Occupancy

When the application starts, it initializes a fixed number of parking bays, all marked available, and pre-populates a handful of them with simulated vehicles and entry times. This gives operators an immediate, realistic baseline view of the facility on first load, rather than an empty grid.

### 7.2 Proximity-Based Slot Allocation

When a vehicle checks in, the system scans the bay records in order and assigns the vehicle to the first bay marked available. This simple, low-complexity approach ensures drivers are routed to the nearest available space rather than wandering the facility, and keeps the allocation logic easy to reason about and extend.

### 7.3 Live Status Feedback

The dashboard continuously recalculates the total number of occupied and available bays from the underlying data model, and displays the result on a persistent status banner. This gives operators immediate, global visibility into the facility's load without needing to inspect individual bays.

### 7.4 Automated Invoicing on Checkout

When a vehicle checks out, the system compares the current time against the recorded entry time to calculate the elapsed parking duration. A fare is then computed by combining a fixed base entry charge with a per-minute usage rate, and an itemized invoice is presented to the driver before the bay is marked available again. This removes the need for manual fare calculation and reduces billing disputes.

## 8. Results

The completed application presents a single-page dashboard showing a live status banner (total available and occupied bays), a vehicle check-in form, a checkout form listing currently occupied bays, and a colour-coded grid map of every bay in the facility. Checking in a vehicle immediately updates the grid and status banner; checking out a vehicle displays a generated invoice with parked duration and total fare, and returns the bay to available status.

## 9. Advantages & Disadvantages

### Advantages

- No Hardware Dependency — the simulated sensor layer allows the full user experience to be demonstrated and tested without installing physical sensors.
- Transparent, Automated Billing — fares are calculated consistently from actual parked time, removing manual estimation errors.
- Simple, Fast Allocation — the nearest-available-bay algorithm is easy to understand, audit, and extend.
- Clear Separation of Concerns — the MVC structure keeps data, presentation, and logic cleanly separated, easing future maintenance.
- Low Barrier to Deployment — built on Flask, the application can be run and demonstrated with minimal setup.

### Disadvantages

- In-Memory Data Only — occupancy data resets whenever the server restarts, since there is no persistent database in the current version.
- Simulated Rather Than Live Sensors — the system does not yet integrate with real hardware sensors, so it does not reflect a physically verified occupancy state.
- Single-Facility Scope — the current design models one facility; a multi-facility, city-wide deployment would require additional architecture.
- No Authentication Layer — the current version does not restrict who can check vehicles in or out, which would need to be addressed for production use.
- Basic Allocation Strategy — always assigning the first available bay does not account for factors like proximity to exits or accessibility needs.

## 10. Conclusion

This project demonstrates a working, automated approach to smart parking management built on a simple Flask web application. By simulating IoT sensor behaviour in an in-memory data model, the system delivers the core user experience of a smart parking facility — live occupancy visibility, automated slot allocation, and transparent, time-based billing — without requiring physical sensor hardware.

The resulting application offers a practical, low-cost blueprint that demonstrates how automation can reduce search time for drivers and administrative overhead for facility operators, while remaining straightforward to extend toward a production-grade, hardware-integrated deployment.

## 11. Future Scope

- Persistent Database Integration — replacing the in-memory data model with a relational database so occupancy and billing history survive server restarts.



- Real Sensor Integration — connecting the platform to physical ultrasonic, infrared, or camera-based occupancy sensors for verified, real-world data.
- Multi-Facility & City-Wide Scaling — extending the architecture to manage multiple parking facilities from a single dashboard.
- Mobile App & Reservation Support — allowing drivers to view availability and reserve a bay in advance from a mobile application.
- Dynamic Pricing — introducing demand-based or time-of-day pricing to better manage peak-hour congestion.
- Payment Gateway Integration — enabling automated digital payment at checkout rather than displaying an invoice only.
- Analytics Dashboard — adding historical occupancy and revenue trends to help operators plan capacity and pricing.
- Role-Based Access Control — introducing operator authentication and permissions for production deployment.